

Concept of Operations

swim-times: Automated Swimmer-Times Retrieval System

College Area Swim Team (CAST)

Prepared for: CAST Coaching Staff and Meet Directors

Prepared by: CAST Technology Working Group

Document identifier: CAST-CONOPS-001

Revision: 1.0

Date: April 21, 2026

Status: Released

Governing standard: IEEE Std 1362-1998 (ConOps Guide)

This document is prepared in accordance with the principles of rational documentation articulated in D. L. Parnas and P. C. Clements, "A Rational Design Process: How and Why to Fake It," IEEE Transactions on Software Engineering, vol. SE-12, no. 2, pp. 251–257, February 1986.

Contents

1	Scope	2
1.1	System identification	2
1.2	Document purpose	2
1.3	System overview	2
2	Referenced Documents	2
3	Current System / Situation (As-Is)	3
3.1	Background	3
3.2	Current problems	3
3.3	Operational environment	3
4	Justification for Change	3
4.1	Deficiencies addressed	3
4.2	Strategic alignment	4
5	Proposed System (To-Be)	4
5.1	System description	4
5.2	High-level capabilities	4
5.3	Operational environment	5
5.4	Interfaces	5
6	Operational Scenarios	6
6.1	Scenario 1 — Post-meet fastest-times update	6
6.2	Scenario 2 — CSV export for PDF report	6
6.3	Scenario 3 — Single-event qualification check	7
6.4	Scenario 4 — Full event history for coaching analysis	7
6.5	Scenario 5 — Cache-and-replay (store then offline)	8
6.6	Scenario 6 — Historical comparison (Leducky 9–10)	8
7	Summary of Impacts	8
7.1	Staffing and workflow	8
7.2	Training	9
7.3	Infrastructure	9
7.4	Security	9
7.5	Compatibility	9
8	Analysis of Alternatives	9
8.1	Alternatives considered	9
8.2	Rationale for chosen approach	10

Scope

System identification

This document is the Concept of Operations (ConOps) for the **swim-times** program, a compiled C utility that retrieves competitive swim times from the USA Swimming data hub and presents them as formatted text or comma-separated-value (CSV) output. The program is implemented as a CWEB literate program whose canonical source is `swim2/swim-times.w`.

Document purpose

This ConOps establishes the operational rationale, usage model, and stakeholder expectations for swim-times so that developers, coaches, and meet directors share a common understanding of what the system does and why it is built the way it is. It is the primary reference for any subsequent requirements specification or design document.

System overview

swim-times accepts command-line options that identify one or more swimmers (Stella Julianna Evans, Kalea Rose Benavente, Kenneth Ray Evans, and Keith Santiago Evans) and one or more USA Swimming event codes (e.g., 100 FR SCY, 200 IM LCM). For each selected swimmer–event pair it queries the live USA Swimming times REST service, retrieves the swimmer’s best recorded times, sorts them fastest-first, and prints either a human-readable table or a CSV stream suitable for downstream processing.

Referenced Documents

- [1] IEEE Std 1362-1998, *IEEE Guide for Information Technology— System Definition— Concept of Operations (ConOps) Document*. IEEE Computer Society, 1998.
- [2] D.L. Parnas and P.C. Clements, “A Rational Design Process: How and Why to Fake It,” *IEEE Transactions on Software Engineering*, vol. SE-12, no. 2, pp. 251–257, February 1986.
- [3] D.E. Knuth, “Literate Programming,” *The Computer Journal*, vol. 27, no. 2, pp. 97–111, 1984.
- [4] D.E. Knuth, *The T_EXbook*. Addison-Wesley, 1984.
- [5] USA Swimming, *Individual Times Search*, <https://data.usaswimming.org/datahub/usas/individual-times-search>. Accessed 2026.
- [6] USA Swimming, *Times API (OpenAPI description)*, <https://times-api.usaswimming.org/swagger>. Accessed 2026.
- [7] `swim2/swim-times.w` — CWEB literate program source (canonical source of record for the swim-times implementation).

- [8] `swim/motivational-time-standards.pdf` — USA Swimming motivational time standards (B, BB, A, ...) used in the companion swim-times PDF report.
- [9] `swim/sdi-agc-time-standards.pdf` — San Diego-Imperial Age-Group Championship qualifying times.

Current System / Situation (As-Is)

Background

CAST coaches and parents currently monitor swimmer progress by visiting the USA Swimming data hub at `data.usaswimming.org`, entering each swimmer's name, and manually reading and transcribing the resulting HTML tables. Times are then typed into a Pandoc Markdown source file (`swim/swim-times.md`) and compiled to PDF.

Current problems

Manual transcription errors.

Copying times by hand from a web browser introduces typographical mistakes; there is no automated verification that the entered time matches what USA Swimming recorded.

Stale data.

The manual workflow is updated intermittently. After any sanctioned meet new times may sit in the USA Swimming database for days before the PDF is regenerated.

Scalability.

The team now tracks four swimmers across thirty-one events each, yielding up to 124 lookup-copy-paste operations per full refresh.

Swimcloud unreliability.

The alternative consumer site `swimcloud.com` is known to contain errors and is therefore excluded from the approved workflow.

Operational environment

The current workflow runs on a macOS workstation equipped with `pandoc` and the `xelatex` PDF engine. No persistent server infrastructure exists; all work is ad hoc and human-driven.

Justification for Change

Deficiencies addressed

The swim-times program eliminates all four problems identified above:

1. Live data is fetched directly from the authoritative USA Swimming times REST service, removing the human copy-paste step entirely.

2. Times are always current because each invocation queries the live API; no stale cache is involved.
3. Four swimmers and thirty-one events can be refreshed in a single command, reducing a multi-hour manual task to a few seconds of CPU time.
4. Swimcloud is bypassed; the only source consulted is `data.usaswimming.org`.

Strategic alignment

Automating time retrieval supports CAST's goal of making accurate, up-to-date performance data available to coaches immediately after every meet. CSV output additionally enables integration with spreadsheets, charting tools, and the team's existing Pandoc-based PDF generation pipeline.

Proposed System (To-Be)

System description

swim-times is a single-binary command-line utility compiled from the CWEB literate program `swim-times.w`. It requires no daemon and no network service beyond HTTPS access to `times-api.usaswimming.org`. An optional local Postgres database (`swim-times`) may be used to persist results and to serve subsequent queries offline.

High-level capabilities

Swimmer selection (`-o` flag).

The operator specifies one or more swimmer keywords (`stella`, `kalea`, `kenny`, `keith`, `ledecky10`, `ledecky12`, `ledecky14`) to restrict output. When no swimmer keyword is given all seven `SWIMMERS[]` entries are processed (the four Evans/Benavente swimmers plus three age-windowed Ledecy entries that share the same person). `ledecky10` retrieves Katie Ledecy's swims as a 9- and 10-year-old (17 March 2006–16 March 2008); `ledecky12` the 11- and 12-year-old window (17 March 2008–16 March 2010); `ledecky14` the 13- and 14-year-old window (17 March 2010–16 March 2012).

Event filtering (`-e` flag).

One or more USA Swimming event codes restrict output to specific events. The flag may be repeated or the codes may be comma-separated. When `-e` is omitted all thirty-one events (fourteen SCY, seventeen LCM) are fetched.

Output modes.

`fastest` emits only the single fastest recorded time per event. `csv` emits a CSV stream with a header row, suitable for import into spreadsheet software or the Pandoc PDF pipeline.

Database persistence (store keyword).

Each retrieved row is also written to the local Postgres database `swim-times` programmatically via the `libpq` client library — one prepared `INSERT ... ON CONFLICT DO NOTHING` per row, keyed on (`swimmer`, `event`, `swim_time`, `swim_date`, `meet`). Re-storing already-present rows is a silent no-op rather than an error. Combine with `csv` to also echo the rows to standard output.

Offline retrieval (offline keyword).

All network calls are skipped; rows are read from the local Postgres `swim-times` database (populated previously by `store`) and printed in the same table or CSV form.

Live authoritative data.

Online runs fetch from the USA Swimming times REST service via anonymous API calls; no local cache is consulted unless `offline` is selected.

Motivational standard annotation.

Each retrieved time includes the USA Swimming motivational standard attained (B, BB, A, ..., or “Slower Than B”) as recorded by USA Swimming.

Operational environment

`swim-times` runs on any POSIX-compatible system with:

- A C99 compiler (`gcc` or `clang`).
- `libcurl` with TLS support (for HTTPS to the times service).
- `CWEB` (`ctangle/cweave`) if the literate source must be retangled or rewoven.
- `pdftex` if the typeset documentation PDF is desired.
- `libpq` (the Postgres C client library) at link and run time when the `store` or `offline` keywords are used. On macOS this ships with the Homebrew `libpq` formula; the project’s `Makefile` autodetects `pg_config` either on `$PATH` or at `/opt/homebrew/opt/libpq/bin/pg-`. No host `psql` client is required. The local Postgres container is still started by `pg.sh start` and the `swim-times` database is created by applying `schema.sql`.

Interfaces**Input.**

Command-line arguments (`-o`, `-e`).

Network.

HTTPS calls to `https://times-api.usaswimming.org/swims/TimesSearch` authenticated by anonymous request headers (no embedded bearer token). Skipped under `offline`.

Database.

Optional local Postgres reached *programmatically* via the `libpq` client library — no `psql` child process and no shell wrapper. `store` runs a prepared `INSERT ... ON CONFLICT DO NOTHING` per row; `offline` runs a parameterised `SELECT ... FROM swim_times`. Connection parameters come from the `SWIM_TIMES_PGCONNINFO` environment variable when set, otherwise the default keyword string `dbname=swim-times user=postgres host=localhost port=5432`.

Standard output.

Formatted plain-text tables or CSV, depending on options.

Standard error.

Diagnostic messages only; no data is written to stderr.

Operational Scenarios

The following scenarios illustrate the primary use cases.

Scenario 1 — Post-meet fastest-times update

Context: It is the evening after a Saturday SCY meet. A coach wants to check whether any of the four tracked swimmers set new personal bests.

1. Coach opens a terminal.
2. Coach runs:

```
./swim-times -o stella,kalea,kenny,keith -o fastest
```

3. swim-times resolves each swimmer's PersonKey from the USA Swimming Persons table.
4. For each swimmer, all thirty-one events are queried and the fastest single time is printed.
5. Total elapsed time is typically under two minutes for all four swimmers.
6. Coach reads the terminal output and identifies any new bests.

Scenario 2 — CSV export for PDF report

Context: A parent wants to regenerate the landscape PDF report (`swim/swim-times.pdf`) for Stella with current times.

1. Parent runs:

```
./swim-times -o stella,csv > stella-latest.csv
```

2. swim-times emits a CSV header followed by one row per recorded time for each of the thirty-one events.
3. Parent (or a script) reads `stella-latest.csv` and updates `swim/swim-times.md`.
4. Parent runs:

```
pandoc swim-times.md -o swim-times.pdf --pdf-engine=xelatex
```

5. The updated PDF is ready.

Scenario 3 — Single-event qualification check

Context: Kalea's coach wants to verify her current 100 Backstroke SCY time ahead of a championship entry deadline.

1. Coach runs:

```
./swim-times -o kalea,fastest -e "100 BK SCY"
```

2. swim-times resolves Kalea's PersonKey, queries the 100 BK SCY event only, and prints the fastest time, date, meet, and motivational standard.
3. Coach compares the printed time against the SDI AGC qualifying standard (from `swim/sdi-agc-time-standards.pdf`) to determine eligibility.

Scenario 4 — Full event history for coaching analysis

Context: Kenneth's coach wants to see his complete 200 Freestyle SCY progression to evaluate training effectiveness.

1. Coach runs:

```
./swim-times -o kenny -e "200 FR SCY"
```

2. swim-times retrieves all recorded 200 FR SCY times for Kenneth, sorts them fastest-first, and prints a table showing time, date, standard, and meet name.
3. Coach reviews the progression, noting improvement trends and peak performance periods.

Scenario 5 — Cache-and-replay (store then offline)

Context: A team analyst wants a snapshot of all swimmers' SCY times that they can re-query repeatedly without hitting the USA Swimming API.

1. Analyst starts the local Postgres container and applies the schema once:

```
../../pg/pg.sh start    ./test-db.sh
```

2. Analyst captures live data into the database:

```
./swim-times -o stella,csv,store
```

Each row is both printed and inserted into `swim_times`.

3. Later, the analyst replays the snapshot offline:

```
./swim-times -o stella,offline,fastest
```

No HTTP request is made; rows are read from Postgres and printed in the same form as an online run.

Scenario 6 — Historical comparison (Ledecky 9–10)

Context: A parent wants to compare Stella's 9-and-under SCY times with Katie Ledecky's swims at the same age.

1. Parent runs:

```
./swim-times -o ledecky10,fastest -e "100 FR SCY"
```

2. swim-times resolves Katie Ledecky's PersonKey, fetches all 100 FR SCY times, filters them to her ninth-through- tenth-birthday window, and prints the fastest.
3. Parent compares against Stella's current best obtained by `./swim-times -o stella,fastest -e "100 FR SCY"`.

Summary of Impacts

Staffing and workflow

No dedicated operator is required. Any team member with access to the compiled binary and a network connection can invoke swim-times. The manual transcription step is eliminated, reducing the time required for a full four-swimmer refresh from approximately one to two hours to under five minutes.

Training

Users need only understand the two flags, `-o` and `-e`, and the swimmer and event keywords. The built-in usage message (printed when no arguments are supplied) provides sufficient reference material for new users. No formal training programme is required.

Infrastructure

No remote infrastructure is required. An optional local Postgres instance (run as a Docker container managed by the `pg.sh` wrapper script) provides persistence and offline replay; absence of that container simply disables the `store` and `offline` keywords without affecting any other capability. The binary must be recompiled if USA Swimming changes its times-service endpoint URLs or authentication scheme. These are maintenance events, not operational events.

Security

The program embeds no credential. The public times service authenticates anonymous callers with three request headers (`Usas-Sub-Id: Anonymous`, `AppName: DataHub`, and a run-time-generated `Device-Id`), and USA Swimming's data hub serves only public competitive records, so all access is read-only to non-sensitive data.

Compatibility

The existing Pandoc-based PDF pipeline (`swim/swim-times.md`) is unchanged. `swim-times` augments rather than replaces it, supplying live data that a human or script pastes into the Markdown source. Future work may automate that final step as well.

Analysis of Alternatives

Alternatives considered

Web scraping via `curl`/HTML parsing.

Directly scraping the HTML rendered by `data.usaswimming.org` was considered. It was rejected because the site renders content via JavaScript after an initial page load; a plain `curl` request returns an empty data table, making reliable extraction fragile and brittle.

Swimcloud API.

Swimcloud (www.swimcloud.com) aggregates USA Swimming data but has known inaccuracies. It was rejected in favour of the authoritative USA Swimming source.

Full JSON parsing library (e.g., `cJSON`, `jansson`).

A third-party JSON parser would have provided a more robust response-processing layer. It was rejected because the REST response format is stable and well-understood, and adding a library dependency would complicate the build for no operational gain.

Python or shell script.

A Python script using `requests` was prototyped. It was set aside in favour of a compiled C program to (a) match the literate programming aesthetic of the CWEB toolchain already used in this project and (b) avoid requiring a Python interpreter on the build host.

Rationale for chosen approach

The chosen design—a CWEB literate C program using `libcurl` and string-scanning—satisfies all operational requirements with minimal external dependencies and produces a self-documenting artefact (the typeset `swim-times.pdf`) that serves simultaneously as executable software and readable reference documentation. This directly embodies the Parnas–Clements principle: the documentation and the implementation are the same source file, presented in the order most useful to the reader rather than the order dictated by the compiler.